



PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PROGRAMMING WITH PYTHON

Exception Handling, File I/O, Modules and Packages

Sensor Data Handling

Lekha A

Computer Applications



PROGRAMMING WITH PYTHON

Sensor Data

- Sensors detect physical changes and convert them into data the program can work with.
- Types of sensors
 - Temperature sensors track heat levels.
 - Light sensors measure brightness.



PROGRAMMING WITH PYTHON

Sensor Data

- Types of sensors
 - Motion sensors detect movement.
 - Pressure, humidity, sound and gas sensors support more advanced environments.
- Each sensor produces readings that describe a specific physical quantity.
- These readings are influenced by environment, component quality and placement.



PROGRAMMING WITH PYTHON

Sensor Data - Measurement units and data types

- Every sensor reading carries a unit that makes the number meaningful.
 - Temperature uses $^{\circ}\text{C}$.
 - Light uses lux.
 - Distance uses centimetres.
 - Motion uses true or false.



PROGRAMMING WITH PYTHON

Sensor Data - Measurement units and data types

- The code stores these values using data types that match the information.
 - Integer values store whole numbers such as counts from a pulse sensor.
 - Float values store continuous measurements such as 25.68 °C.
 - Boolean values store states such as HIGH or LOW from a switch or motion detector.
- Choosing the correct type gives accuracy and avoids incorrect calculations.



PROGRAMMING WITH PYTHON

Sensor Data - Sampling rate and resolution

- Sampling rate shows how often a reading is captured.
 - A slow rate gives fewer data points but uses less memory.
 - A fast rate captures quick changes.
- A light sensor checking ambient light every 10 milliseconds can track screen brightness adjustments in real time.



PROGRAMMING WITH PYTHON

Sensor Data - Sampling rate and resolution

- A temperature sensor in a classroom can use one reading every minute.
- Resolution defines the smallest change the sensor can detect.
 - A sensor with 0.1 °C resolution detects small temperature shifts.
 - A sensor with 1 °C resolution gives coarse data and might hide important variations.



PROGRAMMING WITH PYTHON

Sensor Data - Noise in readings

- Sensor data includes noise.
- Values fluctuate because of interference, hardware limitations or environmental changes.
- Noise appears as small variations even when the source stays steady.



PROGRAMMING WITH PYTHON

Sensor Data - Noise in readings

- A stationary temperature probe may show 25.4, 25.5, 25.3 and 25.6 even under stable conditions.
- This variation does not always reflect real change.
- Smoothing, filtering or averaging is used to reduce noise for clearer analysis.



PROGRAMMING WITH PYTHON

Sensor Data - Analog versus digital signals

- Analog sensors produce continuous values.
 - Light intensity or sound levels fall into this type.
- The microcontroller converts the signal into a number using an ADC.
- Digital sensors send discrete values.



PROGRAMMING WITH PYTHON

Sensor Data - Analog versus digital signals

- Motion detectors give 0 or 1.
- Some temperature sensors send digital data packets.
- Analog values give more detail but need conversion.
- Digital values are simple to read and process but provide less fine-grained information.



PROGRAMMING WITH PYTHON

Sensor Data - Simulation

```
import random
import time

for i in range(10):
    temp_c = 25 + random.uniform(-2, 2)
    print(f"Reading {i+1}: {temp_c:.2f} °C")
    time.sleep(1)
```

Reading 1: 23.98 °C
Reading 2: 26.64 °C
Reading 3: 23.73 °C
Reading 4: 25.79 °C
Reading 5: 26.72 °C
Reading 6: 23.05 °C
Reading 7: 26.91 °C
Reading 8: 25.37 °C
Reading 9: 26.13 °C
Reading 10: 26.19 °C



PROGRAMMING WITH PYTHON

Serial Communication

- Serial communication is used when two devices send data one bit at a time over a wire.
- Example. A laptop connected to an Arduino board with a USB cable.
- Data flows as a timed stream of bits, not as whole messages at once.
- The key idea is that both sides must agree on how fast bits are sent, how they are grouped, and which port is used.



PROGRAMMING WITH PYTHON

Serial Communication – Port Name

- A serial port is the connection endpoint that the program talks to.
- The port name tells the operating system which physical or virtual port is used.
- On Windows
 - Port names look like COM1, COM2, COM3, COM4
 - USB to serial devices are often COM3 or higher
 - When an Arduino or similar board is plugged a new COM port appears



PROGRAMMING WITH PYTHON

Serial Communication – Port Name

- On Linux and macOS
 - Port names look like `/dev/ttyUSB0`, `/dev/ttyUSB1` for USB to serial adapters
 - Some boards use `/dev/ttyACM0`, `/dev/ttyACM1`
 - The `/dev` part is a directory that holds device files



PROGRAMMING WITH PYTHON

Simulate a Virtual Sensor

```
import random
import time
import csv
with open('sensor_readings.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['timestamp', 'temp_c'])
    for i in range(50):
        ts = time.time()
        temp_c = 25 + random.uniform(-2, 2)
        writer.writerow([ts, f"{temp_c:.2f}"])
        print(ts, temp_c)
        time.sleep(0.5)
```



PROGRAMMING WITH PYTHON

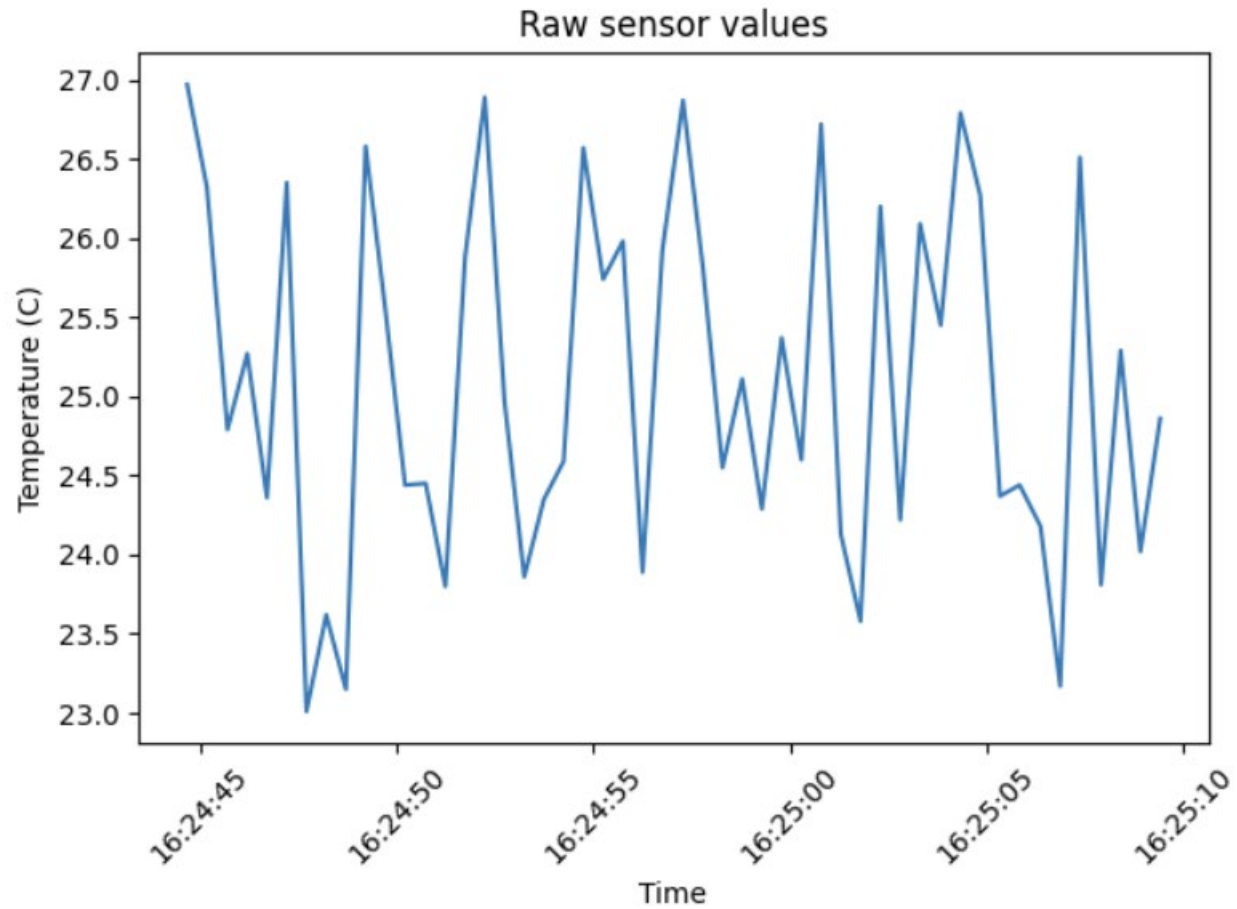
Visualizing sensor readings and interpreting them

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('sensor_readings.csv')
df['timestamp'] = pd.to_numeric(df['timestamp'], errors='coerce')
df['timestamp'] = pd.to_datetime(df['timestamp'], unit='s')
plt.figure()
plt.plot(df['timestamp'], df['temp_c'])
plt.xlabel('Time')
plt.ylabel('Temperature (C)')
plt.title('Raw sensor values')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



PROGRAMMING WITH PYTHON

Visualizing sensor readings and interpreting them





PROGRAMMING WITH PYTHON

Visualizing sensor readings and interpreting them

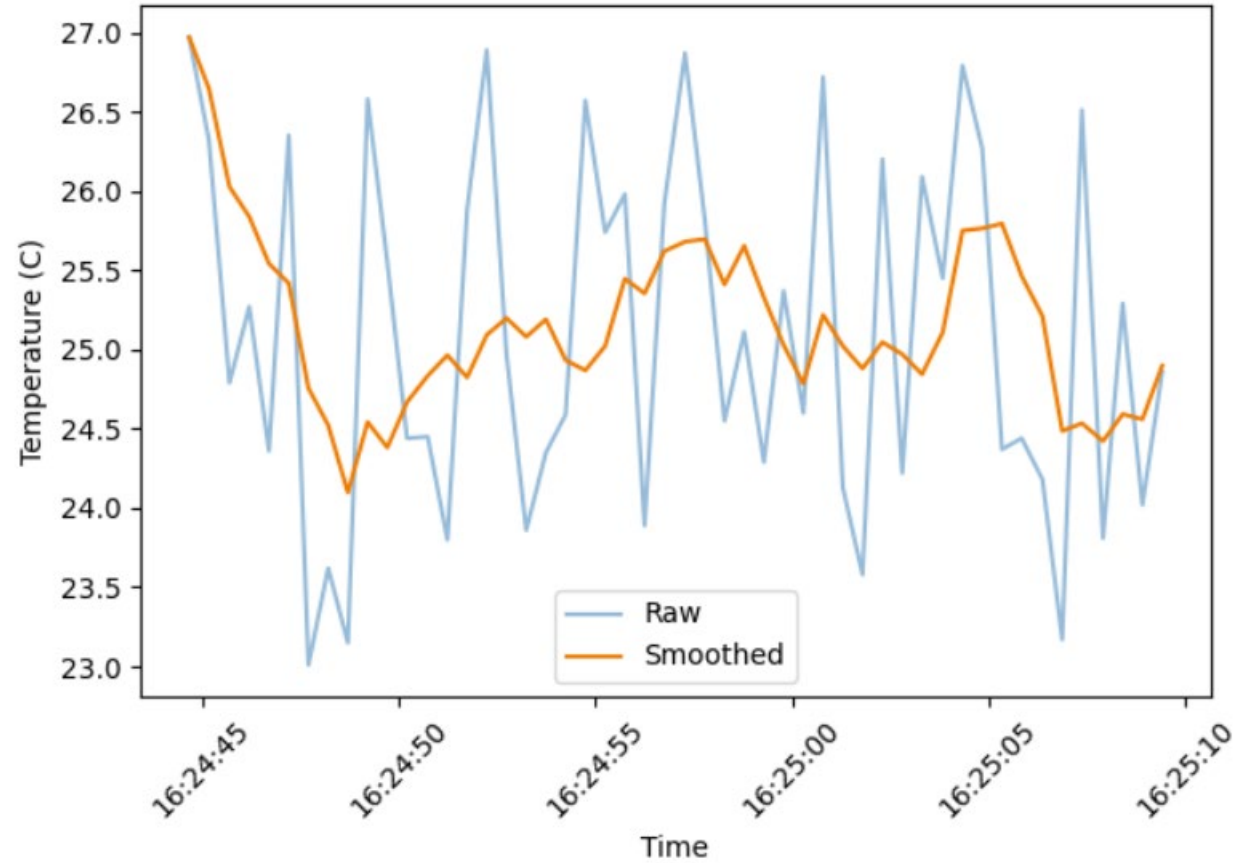
```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('sensor_readings.csv')
df['timestamp'] = pd.to_numeric(df['timestamp'], errors='coerce')
df['timestamp'] = pd.to_datetime(df['timestamp'], unit='s')
df['value'] = pd.to_numeric(df['temp_c'], errors='coerce')
df['value_smooth'] = df['value'].rolling(window=5, min_periods=1).mean()
plt.figure()
plt.plot(df['timestamp'], df['value'], label='Raw', alpha=0.5)
plt.plot(df['timestamp'], df['value_smooth'], label='Smoothed')
plt.xlabel('Time')
plt.ylabel('Temperature (C)')
plt.title('Raw vs smoothed sensor values')
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



PROGRAMMING WITH PYTHON

Visualizing sensor readings and interpreting them

Raw vs smoothed sensor values





PROGRAMMING WITH PYTHON

Visualizing sensor readings and interpreting them

- `df['timestamp'] = pd.to_numeric(df['timestamp'], errors='coerce')`
- `df['timestamp'] = pd.to_datetime(df['timestamp'], unit='s')`
- `df['value'] = pd.to_numeric(df['temp_c'], errors='coerce')`
- `df['value_smooth'] = df['value'].rolling(window=5,
min_periods=1).mean()`
`mid_index = len(df) // 2`



PROGRAMMING WITH PYTHON

Visualizing sensor readings and interpreting them

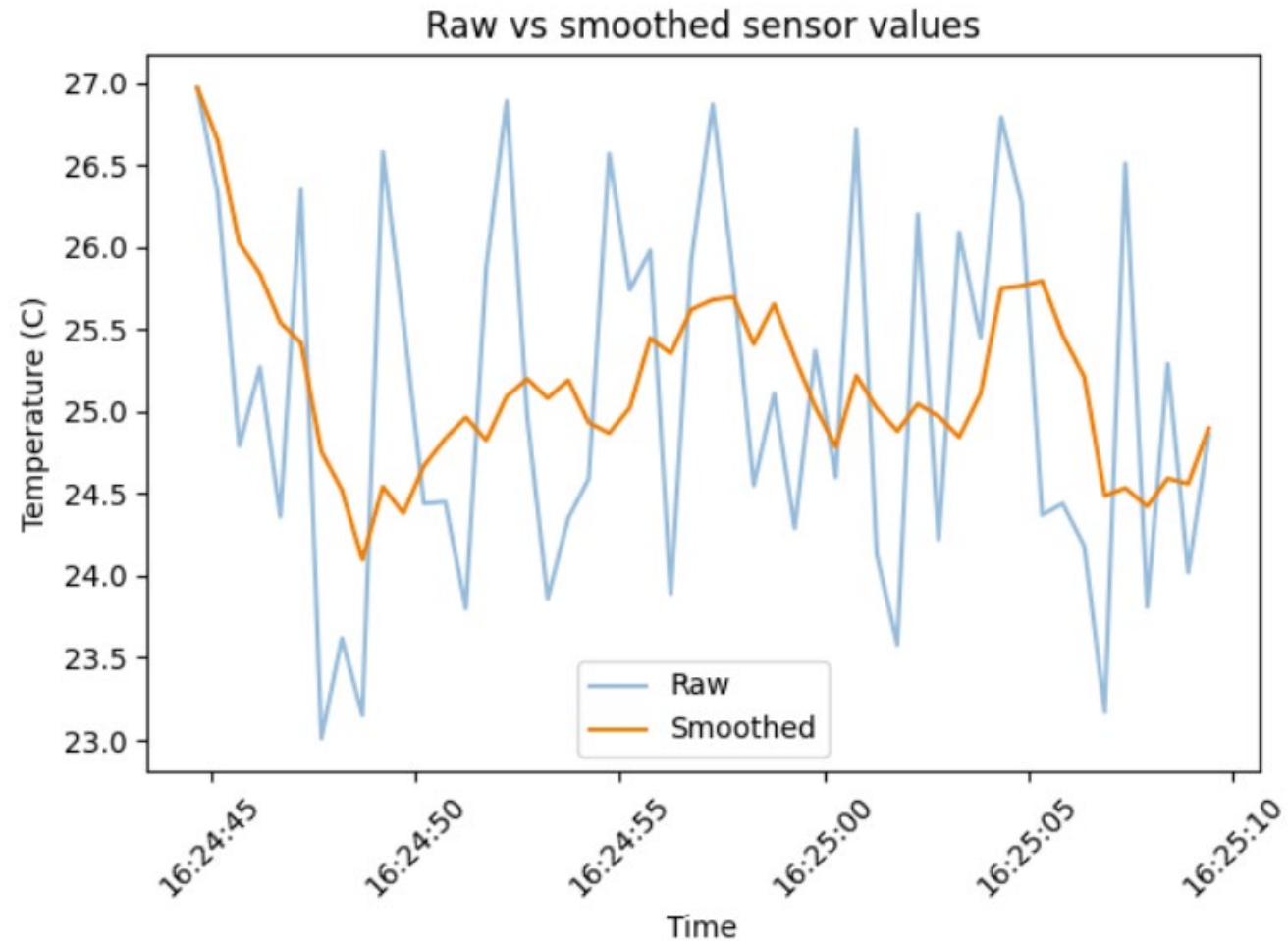
- Compute mid index and means for first and second half
 - `first_half_mean = df['value_smooth'].iloc[:mid_index].mean()`
 - `second_half_mean = df['value_smooth'].iloc[mid_index:].mean()`
 - If the second half mean is higher, the temperature increased over time.
 - If the second half mean is lower, the temperature decreased.
 - If both means are close, the environment stayed stable.



PROGRAMMING WITH PYTHON

Visualizing sensor readings and interpreting them

First half mean: 25.193246666666667
Second half mean: 25.125680000000003





THANK YOU

Lekha A

Computer Applications